



UNIVERSIDADE FEDERAL DE MINAS GERAIS

INTRODUÇÃO A SISTEMAS LÓGICOS

Detector de Números Primos de 4 bits

Autor:

Rodrigo de Freitas Lanaza

Matrícula: 2025051446

23 de maio de 2026

Conteúdo

1	Introdução	2
2	Decisões de Implementação do Circuito	2
3	Tabela Verdade	2
4	Forma Canônica	2
4.1	Soma de Mintermos	3
4.2	Produto de Maxtermos	3
5	Mapa de Karnaugh	3
5.1	4.1. Mapa de Karnaugh	3
5.2	4.2. Mapa de Karnaugh (Primos Essenciais)	4
6	Minimização	4
6.1	Classificação dos Implicantes	4
6.2	Função Minimizada (Produto de Somas - PoS)	4
7	Diagrama Lógico do Circuito	4
8	Códigos Fonte em Verilog	5
8.1	7.1. Circuito Principal (<code>design.sv</code>)	5
8.2	7.2. Ambiente de Simulação (<code>testbench.sv</code>)	6
9	Formas de Onda do Testbench	6

1 Introdução

Em sistemas digitais, é frequente a necessidade de identificar propriedades específicas de valores numéricos representados em formato binário. Um exemplo clássico desta aplicação é a identificação de números primos, definidos como números inteiros maiores que 1 que são divisíveis apenas por 1 e por si mesmos.

O objetivo deste trabalho prático é projetar e implementar um circuito lógico combinacional capaz de detetar se um dado número binário de 4 bits representa um número primo. A implementação deve ser realizada utilizando a linguagem de descrição de hardware Verilog, adotando um estilo de modelação estrutural.

O circuito projetado recebe como entrada um valor binário de 4 bits ($N[3 : 0]$), o que possibilita representar valores decimais num intervalo de 0 a 15. A saída do circuito consiste num único bit (P), tal que $P = 1$ indica que a entrada corresponde a um número primo, e $P = 0$ indica que o número não é primo. Dentro do limite de 4 bits, os valores que ativam a saída do sistema são: 2, 3, 5, 7, 11 e 13.

2 Decisões de Implementação do Circuito

Para montar o circuito que detecta números primos, seguindo as definições do trabalho, montei um circuito combinacional e utilizei Verilog no estilo estrutural. Não foi utilizado descrições comportamentais completas, `if` e `always`). O código Verilog e o diagrama lógico foi construído apenas conectando portas lógicas básicas **AND**, **OR** e **NOT**. Para que o circuito ficasse o menor possível, simplifiquei a função usando o Mapa de Karnaugh usando **Pos**, Product of Sums.

Utilizei em todas as portas lógicas, portas com duas entradas apenas. Por causa dessa limitação, quando a equação exigia somar três variáveis ao mesmo tempo, liguei duas portas OR em sequência. Já para a multiplicação final de todos os termos, organizei as portas AND, juntando os sinais aos poucos, de dois em dois, até chegar ao resultado final de forma limpa e organizada.

3 Tabela Verdade

A Tabela verdade abaixo apresenta a especificação completa do comportamento esperado para o circuito combinacional. A função de saída (P) assume o valor lógico de 1 exclusivamente para as entradas cujo valor decimal seja um número primo.

Decimal	Binário ($N[3 : 0]$)	Saída (P)
0	0000	0
1	0001	0
2	0010	1
3	0011	1
4	0100	0
5	0101	1
6	0110	0
7	0111	1
8	1000	0
9	1001	0
10	1010	0
11	1011	1
12	1100	0
13	1101	1
14	1110	0
15	1111	0

4 Forma Canônica

A forma canônica descreve a função lógica diretamente a partir da tabela verdade. Pode ser expressa como a Soma de Mintermos (focando nas linhas onde a saída é 1) ou como o Produto de Maxtermos (focando nas linhas onde a saída é 0).

Considerando o vetor de entrada $N[3 : 0]$ com as variáveis N_3 , N_2 , N_1 e N_0 (sendo N_3 o bit mais significativo), sabemos que a saída $P = 1$ para os seguintes valores decimais: 2, 3, 5, 7, 11 e 13.

4.1 Soma de Mintermos

Utilizando a notação padrão para a soma de mintermos, a função lógica do circuito é definida por:

$$P(N_3, N_2, N_1, N_0) = \sum m(2, 3, 5, 7, 11, 13)$$

A expressão booleana expandida (Soma de Produtos) correspondente a estes mintermos é escrita como:

$$P = (\overline{N_3}\overline{N_2}N_1\overline{N_0}) + (\overline{N_3}\overline{N_2}N_1N_0) + (\overline{N_3}N_2\overline{N_1}N_0) + (\overline{N_3}N_2N_1N_0) + (N_3\overline{N_2}N_1N_0) + (N_3N_2\overline{N_1}N_0)$$

4.2 Produto de Maxtermos

Em contrapartida, avaliando os casos onde a saída não representa um número primo ($P = 0$), obtemos a notação em produto de maxtermos:

$$P(N_3, N_2, N_1, N_0) = \prod M(0, 1, 4, 6, 8, 9, 10, 12, 14, 15)$$

A expressão booleana expandida (Produto de Somas) correspondente a estes mintermos é escrita como:

$$\begin{aligned} P = & (N_3 + N_2 + N_1 + N_0) \cdot (N_3 + N_2 + N_1 + \overline{N_0}) \\ & \cdot (N_3 + \overline{N_2} + N_1 + N_0) \cdot (N_3 + \overline{N_2} + \overline{N_1} + N_0) \\ & \cdot (\overline{N_3} + N_2 + N_1 + N_0) \cdot (\overline{N_3} + N_2 + N_1 + \overline{N_0}) \\ & \cdot (\overline{N_3} + N_2 + \overline{N_1} + N_0) \cdot (\overline{N_3} + \overline{N_2} + N_1 + N_0) \\ & \cdot (\overline{N_3} + \overline{N_2} + \overline{N_1} + N_0) \cdot (\overline{N_3} + \overline{N_2} + \overline{N_1} + \overline{N_0}) \end{aligned}$$

5 Mapa de Karnaugh

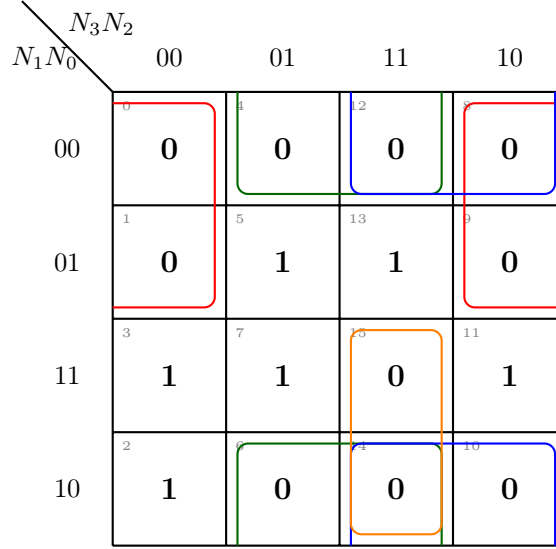
Para obter a forma simplificada em Produto de Somas (PoS), mapeamos os maxtermos da função, que correspondem às combinações binárias cuja saída é 0 (números não primos).

Abaixo estão apresentados os dois Mapas de Karnaugh estruturados de forma quadrada com a contagem disposta estritamente na vertical (de cima para baixo). O primeiro mapa exibe os valores puros do circuito e o segundo ilustra os laços dos agrupamentos. Os números menores em cinza indicam o valor decimal equivalente de cada célula.

5.1 4.1. Mapa de Karnaugh

		N_3N_2			
		N_1N_0	00	01	11
	00	⁰ 0	⁴ 0	¹² 0	⁸ 0
	01	¹ 0	⁵ 1	¹³ 1	⁹ 0
	11	³ 1	⁷ 1	¹⁵ 0	¹¹ 1
	10	² 1	⁶ 0	¹⁴ 0	¹⁰ 0

5.2 4.2. Mapa de Karnaugh (Primos Essenciais)



6 Minimização

A partir da análise dos agrupamentos máximos de zeros isolados no mapa anterior, podemos classificar e mapear os termos simplificados.

6.1 Classificação dos Implicantes

- **Implicantes Primos Encontrados:** (0,1,8,9), (4,6,12,14), (8,10,12,14), (14,15) e (0,4,8,12).
- **Implicantes Primos Essenciais:** Aqueles que cobrem elementos de forma única e exclusiva. São os responsáveis por compor a resposta final.
 - **Laço Vermelho (0, 1, 8, 9):** Essencial para cobrir exclusivamente os zeros das células 1 e 9. Termo: $(N_2 + N_1)$
 - **Laço Verde (4, 6, 12, 14):** Essencial para cobrir exclusivamente o zero da célula 6. Termo: $(\overline{N_2} + N_0)$
 - **Laço Azul (8, 10, 12, 14):** Essencial para cobrir exclusivamente o zero da célula 10. Termo: $(\overline{N_3} + N_0)$
 - **Laço Laranja (14, 15):** Essencial para cobrir exclusivamente o zero da célula 15. Termo: $(\overline{N_3} + \overline{N_2} + \overline{N_1})$
- **Implicantes Primos Não Essenciais:** O grupo formado pelas células (0, 4, 8, 12) é considerado não essencial, uma vez que todos os seus zeros já foram completamente absorvidos e cobertos pelos demais grupos essenciais. Por esse motivo, ele foi descartado e não aparece desenhado no mapa.

6.2 Função Minimizada (Produto de Somas - PoS)

Realizando a operação distributiva AND sobre os termos extraídos unicamente dos grupos essenciais, chegamos à expressão final simplificada para o circuito:

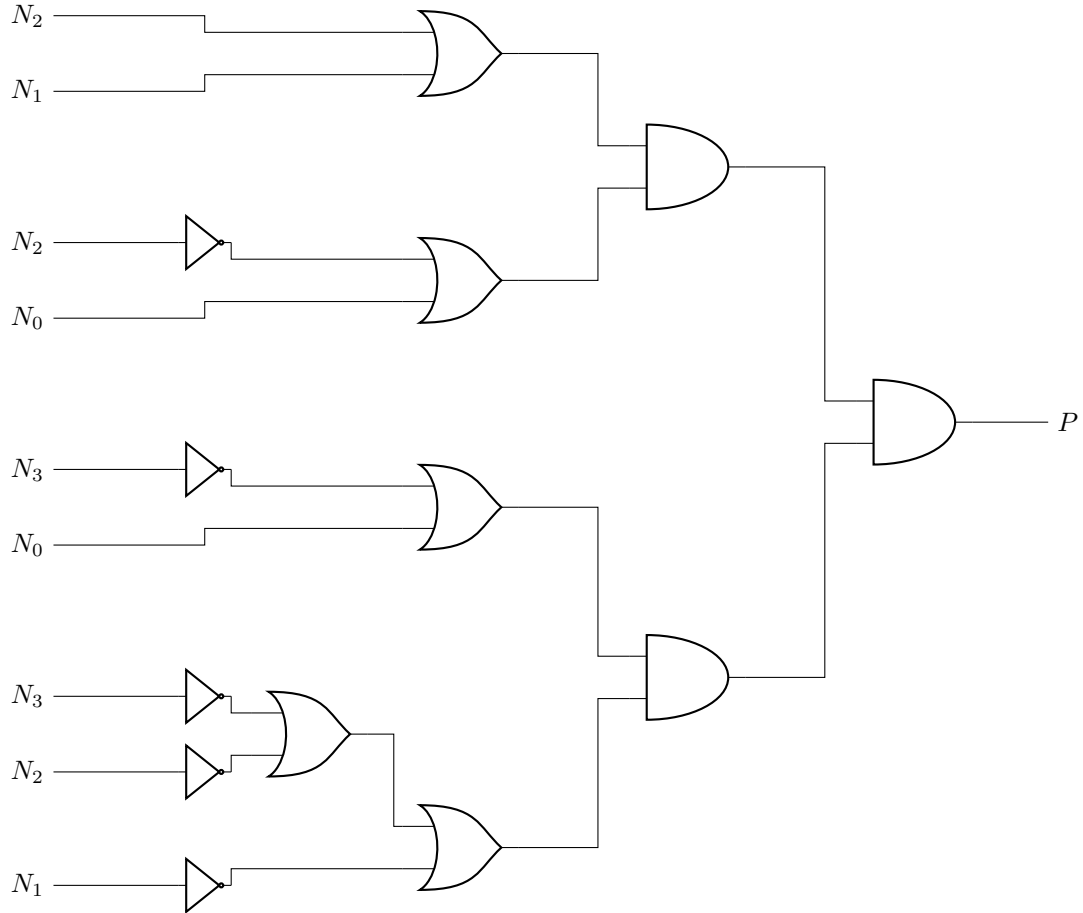
$$P(N_3, N_2, N_1, N_0) = (N_2 + N_1) \cdot (\overline{N_2} + N_0) \cdot (\overline{N_3} + N_0) \cdot (\overline{N_3} + \overline{N_2} + \overline{N_1})$$

7 Diagrama Lógico do Circuito

O diagrama lógico a seguir representa a implementação estrutural do circuito combinacional, projetado diretamente a partir da nossa equação minimizada em Produto de Somas (PoS):

$$P(N_3, N_2, N_1, N_0) = (N_2 + N_1) \cdot (\overline{N_2} + N_0) \cdot (\overline{N_3} + N_0) \cdot (\overline{N_3} + \overline{N_2} + \overline{N_1})$$

Este circuito foi desenhado utilizando portas **AND**, **OR** e **NOT**. Para o termo da soma com 3 variáveis, duas portas **OR** foram ligadas em cascata. Para o produto final dos 4 termos, foi utilizado três portas **AND** de 2 entradas. Entradas negadas foram feitas com 3 portas **NOT**, uma em cada entrada.



8 Códigos Fonte em Verilog

Abaixo estão apresentados os códigos desenvolvidos em Verilog estrutural. O primeiro bloco exibe a implementação do circuito lógico e o segundo bloco exibe o ambiente de testes (testbench) utilizado para a validação.

8.1 7.1. Circuito Principal (design.sv)

```

1 module prime_detector(
2     input wire [3:0] N,
3     output wire P
4 );
5
6     wire not_n3, not_n2, not_n1;
7     wire or1_out, or2_out, or3_out, or4_out, or5_out;
8     wire and1_out, and2_out;
9
10    not inv_n3 (not_n3, N[3]);
11    not inv_n2 (not_n2, N[2]);
12    not inv_n1 (not_n1, N[1]);
13
14    or soma1 (or1_out, N[2], N[1]);
15    or soma2 (or2_out, not_n2, N[0]);
16    or soma3 (or3_out, not_n3, N[0]);
17    or soma4 (or4_out, not_n3, not_n2);
18    or soma5 (or5_out, or4_out, not_n1);
  
```

```

19
20     and mult1 (and1_out, or1_out, or2_out);
21     and mult2 (and2_out, or3_out, or5_out);
22     and mult3 (P, and1_out, and2_out);
23
24 endmodule

```

8.2 7.2. Ambiente de Simulação (testbench.sv)

```

1 module prime_detector_testbench;
2
3     reg [3:0] N_testbench;
4     wire P_testbench;
5     integer i;
6
7     prime_detector_testbench prime_detector (.N(N_testbench), .P(P_testbench));
8
9     initial begin
10         $dumpfile("dump.vcd");
11         $dumpvars(1, prime_detector_testbench);
12
13         $display("-----");
14         $display(" time (ns) |          input          | Decimal | Prime?");
15         $display("-----");
16         $monitor("   %4d   |   %4b   |   %2d   |   %b",
17                 $time, N_testbench, N_testbench, P_testbench);
18
19         for (i = 0; i < 16; i = i + 1) begin
20             N_testbench = i;
21             #10;
22         end
23
24         #10;
25
26         $finish;
27     end
28
29 endmodule

```

9 Formas de Onda do Testbench

Para validar o funcionamento do circuito projetado, foi gerado o gráfico de formas de onda (waveforms). O gráfico mostra a variação do sinal de saída (P_testbench) e as entradas de todos os valores binários possíveis (de 0 a 15) no barramento de entrada (N_testbench).

Nota-se que a saída é verdadeira perfeitamente em sincronia com as entradas equivalentes aos números primos (2, 3, 5, 7, 11 e 13).

